Evaluating a SQL explainer with a golden set and an LLM judge

Project 1 report · CS496 · 2026-10-21 · repo `project1-grader` · final system: prompt v3 on `gpt-5-nano`, judge v1 on `gpt-5-mini` (OpenAI)

> **Summary.** On 101 held-out test queries, system v3 is rated good on **78.0/101 (77.2%)**, the mean of three runs (75.2–79.2%), up from about 69% for the baseline prompt on dev. That score comes from an LLM judge that agrees with our golden labels on **65/101 test items (64.4%, κ 0.33)**, against 90% on dev. The gap comes from fixing dev labels where the judge disagreed, plus a judge that is too strict. The judge catches 93% of bad answers and shows no position or length preference. Running the system costs **$0.19 per 1,000 requests**.

1. The task, and why exact match isn't enough

Input: a SQLite query over a five-table shop schema. Output: a plain-language explanation for a reader who doesn't know SQL. An explanation is **good** only if it is correct, complete (every filter, grouping, aggregate, limit and sort direction), invents nothing, and is clear to that reader. There are many right answers and one wrong word can break one, so neither exact match nor word overlap works:

> `SELECT DISTINCT price FROM products ORDER BY price DESC LIMIT 1 OFFSET 1;` (c-q117)
> **Good A:** "...orders them from most expensive to least, skips the top (most expensive) price, and then returns the next one."
> **Good B** (system v3): "The query returns the second-highest distinct price... skips the very first (highest) price, and then returns the next one. If there aren't at least two distinct prices, it returns no rows."
> **Wrong** (planted): identical to A except "skips the top two (most expensive) prices".
> Word overlap (Jaccard): A vs B **0.42**, both good. A vs Wrong **0.97**, one is wrong. Overlap would rank the wrong answer highest.

2. The golden set

Queries. 161 queries: 50 from Project 0 and 111 new ones aimed at traps (inner vs left joins, NULLs, `DISTINCT`, window functions, missing `ORDER BY`). All were checked against the database. They are split 60 dev / 101 test, stratified by difficulty (45 easy, 59 medium, 57 hard).

Candidates. Each query has one blind candidate explanation: 73 baseline (system model), 32 from a second, weaker model, and 56 with a planted error. The planted errors are six types (wrong_filter 10, missing_condition 10, wrong_join_semantics 9, wrong_aggregation 9, invented_detail 9, jargon 9). We mixed sources so that bad answers include both natural and known errors. Labellers never saw the source.

Guide and labelling. `data/LABELLING_GUIDE.md` (v3) defines the four criteria and twelve decision rules. The rules people disputed most: silence about an edge case is fine but implying the wrong behaviour is not (3.1); sort direction is required (3.3); "all" must mean all (3.11); and "lists every X" is wrong over an inner join (3.12). With instructor approval, two independent Claude Opus 5.5 subagents labelled everything from blind packets.

Agreement (claude-a vs claude-b)	Result
Raw agreement / Cohen's κ	157/161 (97.5%, CI 93.8–99.0) / 0.947
Confusion (a × b): good-good / good-bad / bad-good / bad-bad	99 / 2 / 2 / 58
By difficulty: easy / medium / hard	97.8% / 98.3% / 96.5%
By source: baseline / weak / perturbed (κ)	0.84 / 0.65 / 0.88
Dropped / adjudicated / overridden after agreement	0 / 4 / 17
Final golden set (dev; test)	82 good / 79 bad (24/36; 58/43)

High agreement between two copies of one model shows consistency, not correctness: both labellers passed 4 of 55 planted errors. The 4 disagreements were adjudicated, which added rules 3.10 and 3.11 in guide v2. We then made 17 overrides where both labellers agreed but broke a guide rule, each citing the rule in `data/labels/overrides.jsonl`. Rule 3.12 came from the judge's dev disagreements and was applied by a sweep over all 161 items, which changed 10 labels. Later fixes (c-q108, c-q095, c-q145, c-q130, c-q107) were made **on dev only**, which matters in §4.

3. The harness

`python -m harness.run --split test --system v3` runs the system on every query, grades each answer with the judge, and writes one row per item: input, output, grade, reason, failed criteria, tokens, latency, cost and any error. It also

appends a line to `results/history.csv`, and if an earlier run exists it lists exactly which items changed grade. Every model call must answer in a fixed JSON schema (strict `json_schema`, validated again locally). A reply that doesn't match is retried once, then recorded as an error row, never parsed out of free text and never counted as a zero. Responses are cached by exact request, so interrupted runs resume for free. A mock backend and 12 unit and end-to-end tests run offline. We moved from Groq to OpenAI mid-project (an instructor-provided key; Groq's daily cap blocked steps 6–9). Only the client and model table changed, plus a flag because gpt-5 models reject `temperature`.

4. The judge: reliability report

The judge is prompt v1 on gpt-5-mini. Prompt v2, which writes guide v3's rules into the prompt, tied it on dev at 54/60 but marked more good answers bad (precision 36/42 vs 34/38), so we kept v1.

Judge vs golden labels	Dev (60)	Test (101)
Agreement (95% CI)	54/60 (90.0%)	**65/101 (64.4%, 54.6–73.0)**
Cohen's κ	0.79	**0.33**
Recall of bad (human-bad also judged bad)	34/36	40/43 (93.0%)
Precision of bad (judge-bad that humans call bad)	34/38	40/73 (54.8%)
Agreement: easy / medium / hard	94 / 95 / 86%	79 / 78 / **39%**

Why test is so much lower. (1) *Label fixing was one-sided.* Dev labels were corrected where the judge exposed rule violations, but test labels were not, and can't be now without relabelling after seeing the judge's verdicts. Several test disagreements follow the patterns we fixed on dev: "purchased/bought/sold" with no status filter (c-q035, c-q037, c-q136, c-q146) and "every order" over an inner join (c-q050, c-q156). (2) *The judge is too strict.* About 23 of its 33 false "bad" calls demand details that rules 3.1–3.2 say may be left out: rows merged by `GROUP BY name` (c-q026, c-q029, c-q038, c-q044), exclusions the explanation never denies (c-q018, c-q043, c-q123, c-q148), unstated default ascending sort (c-q060, c-q118), rounding, and currency (c-q053). **Test agreement (64%) is the honest reliability number.**

Planted errors caught on test: invented_detail 7/7, missing_condition 7/7, wrong_join_semantics 6/6, wrong_filter 5/5, wrong_aggregation 4/4, jargon 5/6. The judge almost never lets a planted error through, but its "bad" verdicts are only right about half the time.

Position bias (pairwise judge, original vs alternative, both orders, test)

The judge gave the same verdict in both orders on 87/101 pairs (86.1%). Only 4/101 picked the same slot both times; the other flips switched between a tie and a winner. First slot won 61/202 (30.2%), second 63/202 (31.2%), tie 78/202 (38.6%). On 35 pairs with a known better answer, it picked the unbroken original in both orders 31/35 (88.6%) and the broken one in either order 2/35. **No slot preference.**

Verbosity bias (the same answer padded to about 2.8× its length)

	Template padding	LLM padding
Grades changed	12/101 (11.9%)	19/101 (18.8%)
Judge-caught human-bad answers rescued by padding	**2/40 (5.0%)**	5/40 (12.5%)*
Human-good answers penalised by padding	5/25 (20.0%)	5/25 (20.0%)
Pairwise: padded / original / tie	25 / **35** / 142	60 / 31 / 111*

Template padding adds a fixed preamble and generic SQL facts, so the content is provably unchanged. On that clean test, padding rarely rescues a bad answer and the judge slightly prefers the original: **no reward for length**. *The LLM-padding numbers overstate the bias. Spot checks show the pad model rewrote jargon into plain words (c-q027, c-q086, c-q089) and dropped an invented detail (c-q150), so some padded answers really were better.

Where the judge fails

- **Too strict on hard queries**: 14/36 agreement on test. It treats "for each X, it computes…" as a claim that every X appears (c-q030, c-q112 with v2; c-q114).
- **Fails answers for things the guide allows them to leave out**: c-q083 (rows repeated per order), c-q126 (first month's NULL), c-q061 (NULL countries).
- **Misses clause-by-clause jargon**: c-q140 (dev) and c-q120 (test), for both prompt versions and both providers.
- **Borderline on default sort direction** (c-q014, c-q060, c-q118). The guide doesn't settle whether "sorted by year" has to say ascending.

5. The system: score history

gpt-5-nano and gpt-5-mini don't accept `temperature`, so runs aren't repeatable. Two identical v1 dev runs scored 43 and 40, with 19 items changing grade. We therefore count a version as better only if it beats the baseline across reruns.

Version	One change	Dev (judge v1)	Effect
v1	Baseline: "explain in plain words"	43/60, 40/60	7 of 17 failures were row scope, 3 were row order
v2	Be exact about which rows: an inner join drops unmatched rows; no invented status filter	43/60	No change: 9 items better, 9 worse, the same as noise. Fixed join wording but started inventing row orders (c-q075, c-q127)
v3	v2 + describe order only from `ORDER BY`, with direction	**51/59, 47/59**	Both runs beat both v1 runs (about +8 items). Fixed c-q046, c-q107, c-q154, c-q075, c-q127

Test, v3, three runs: 78, 76, 80 of 101, so a mean of **77.2%** (range 75.2–79.2%; per-run CI about ±8 points). By difficulty, the mean was easy 90%, medium 82%, hard 62% (20–26 of 36, the most variable). For test the system's token limit went from 1,024 to 2,048 because v3 hit the cap on dev q159. Read this score together with §4: the judge fails about a third of human-good test answers, so true quality is likely somewhat higher than this score.

6. Cost model

From the third test run, at OpenAI list prices; reasoning tokens are billed as output.

	System (gpt-5-nano)	Judge (gpt-5-mini)
Tokens in / out per item	457 / 408	950 / 913
Cost per item	$0.000186	$0.00206
Latency p50 / p95	3.7 s / 5.5 s	10.0 s / 18.6 s

Per 1,000 requests: **$0.19** for the system alone, $2.25 with every answer judged, and **$0.39** with a 10% judged sample for monitoring. Assuming 500 requests/day (our assumption, not measured), that's about **$5.90/month**, and about **$588/month at 100×**. The judge costs about 11 times more per item than the system, so judge a sample, not every answer. Our OpenAI spend for the whole evaluation was about $4 (estimated: $2.24 is logged in run, judge and position-bias files, and the two verbosity checks made about 600 more judge calls). Groq's free tier had been enough for candidate generation and the early judge work, but not for steps 6–9, which is why we switched.